

Tool-Action Batching Is Not a Free Lunch for Terminal Agents: A Controlled Study of Interaction Boundaries, Cost, and Reliability

Sid Ravi Sidharth Santham
Interhuman AI
{sid,siddsantham}@gmail.com

Abstract

Iterative ReAct-style LLM coding agents execute repository tasks through alternating turns of model reasoning and tool invocation. For short, dependent tool actions (e.g., `grep` $\rightarrow$ `read` $\rightarrow$ `bash` $\rightarrow$ `verify`), stopping for a full model turn after every intermediate result adds latency, API overhead, and context growth. We introduce `batched-queue`, a typed action queue for Pi and OpenCode coding agents that executes sequences of dependent repository actions with persistent shell state, fast-fail semantics, and typed variable bindings in a single tool turn. We perform a controlled empirical study comparing native tool execution with batched execution across three benchmark layers: (1) model-free deterministic mechanics (D1–D8), (2) controlled fixture scenarios (H1–H24), and (3) a full paired evaluation on Terminal-Bench 2.1 across all 88 tasks (178 containerized episodes) using `gpt-5.6-luna`. On deterministic and controlled fixtures, explicit batching compresses interaction turns by 50%–67% while maintaining 100% pass rates. However, on realistic Terminal-Bench tasks under neutral tool availability, batch availability does not improve task resolution (58/88 vs 65/88 pass@1, exact McNemar $p = 0.189$) and increases median agent execution time (+15.33 s). We find that tool-action batching is an interface optimization bounded by task dependency structure, model adoption choices, and error propagation—not a universal agent reliability upgrade.

1 Introduction

Large Language Model (LLM) terminal agents typically operate in a single-action Read-Eval-Print Loop (REPL) [8]. When diagnosing a bug or refactoring code, an agent often knows a sequence of dependent actions in advance: find a symbol, read the relevant file segment, run a targeted unit test, and check the output. In standard agent architectures, each step requires returning control to the LLM, triggering a round-trip network call, prompt re-tokenization, and model inference.

While parallel tool calling [6, 5] handles **independent** tool actions (e.g., reading three files simultaneously), it cannot execute **dependent** actions where step k requires the output of step $k - 1$ (e.g., using a file path discovered by `grep` to read lines).

To address this gap, we present `batched-queue`, a lightweight tool extension for Pi and OpenCode agents. `batched-queue` enables the driver model to submit a typed pipeline of up to N dependent repository actions (`read_lines`, `grep_pattern`, `execute_bash`, `apply_diff`) executed sequentially in a single tool turn without intermediate LLM calls. The queue preserves persistent shell state across bash actions, supports fast-fail execution on non-zero exit codes or syntax errors, and enables variable interpolation via typed `bindTo` results (e.g., `$marker`).

Research Questions. We evaluate `batched-queue` across five questions:

- **RQ1 (Interaction Boundaries):** Does batching reduce LLM-tool turn boundaries on dependent terminal workflows?
- **RQ2 (Correctness & Invariants):** Does batching preserve execution safety, shell state, and task correctness?
- **RQ3 (External Transfer):** Do fixture gains transfer to realistic terminal workloads on Terminal-Bench 2.1?
- **RQ4 (Cost & Latency):** What are the token, wall-clock, and cost trade-offs of batch availability?

- **RQ5 (Workload Boundaries):** Which task categories benefit from action batching, and which suffer?

Contributions.

1. **Typed Action Queue Architecture:** A formal specification and implementation of a low-latency dependent action queue with persistent shell state, variable binding, workspace boundaries, and fast-fail halt metadata.
2. **Multi-Layer Evaluation Protocol:** An evaluation suite comprising model-free deterministic mechanics (D1–D8), controlled fixture scenarios (H1–H24), and an official Harbor adapter for Terminal-Bench 2.1.
3. **Complete Terminal-Bench 2.1 Empirical Result:** A full paired evaluation on all 88 tasks of Terminal-Bench 2.1 (178 episodes), establishing that tool-action batching is a bounded interface mechanism rather than a universal reliability win.

2 System Architecture: batched-queue

batched-queue provides a single tool `batch_queue` accepted by Pi and OpenCode coding agents. Rather than replacing the driver LLM, it exposes a typed mini-pipeline interface.

2.1 Action Schema and Execution Flow

A batch payload consists of an ordered list of typed actions $A = [a_1, a_2, \dots, a_k]$ where $k \leq 10$. The supported action discriminators are:

- `read_lines(path, startLine, endLine, bindTo)`: Reads a line range from a workspace-bounded file.
- `grep_pattern(pattern, path, glob, bindTo)`: Executes ripgrep over workspace files with match limits.
- `execute_bash(command, timeoutMs, bindTo)`: Executes bash commands in a persistent subshell.
- `apply_diff(path, oldText, newText)`: Applies exact string/diff edits with syntax validation.

2.2 Persistent Shell State and Result Bindings

To support dependent workflows, the executor maintains a single `PersistentShell` session per agent turn. Environment variables exported in step a_1 (e.g., `export ENV=prod`) and working directory changes (`cd nested`) persist across subsequent actions $a_2 \dots a_k$.

Furthermore, actions can declare a `bindTo: "varName"` parameter. The raw stdout or text output of the action is captured into a session variable map. Subsequent string fields in later actions can reference `$varName`, which the runner interpolates before action execution.

2.3 Safety Invariants and Fast-Fail Semantics

The queue enforces three safety mechanisms:

1. **Workspace Boundaries:** All file paths are resolved relative to the workspace root and validated against path escape (`.` `..`). Reads outside the workspace are rejected.
2. **Syntax and Diff Validation:** `apply_diff` parses TypeScript/JSON ASTs pre- and post-application; patches introducing syntax errors fast-fail the batch.
3. **Fast-Fail Halting:** If any action a_i fails (non-zero exit code, command timeout, or syntax error), execution immediately halts. Remaining actions $a_{i+1} \dots a_k$ are discarded, preventing cascading mutations.

3 Experimental Design

3.1 Benchmark Layers

1. Deterministic Mechanics (D1–D8): A model-free test suite running against temporary repository fixtures. It validates executor equivalence, shell persistence, binding resolution, fast-fail execution, timeout recovery, workspace safety, diff verification, and independent-read baselines without LLM noise or API costs.

2. Controlled Fixture Suite (H1–H24): A suite of 24 synthetic scenarios testing specific agent interaction patterns (e.g., loader trace, failure observation, shell variable reuse). Scenarios use fixture-owned oracles checking hidden proof specifications and deterministic re-verification commands.

3. Terminal-Bench 2.1 (External Evaluation): The official dataset containing 89 containerized Linux tasks from harbor-framework/terminal-bench-2-1 (dataset commit 36d417f5). Tasks range from system administration and web server setup to C++/Python debugging and database recovery.

3.2 Paired Evaluation Protocol and Controls

For Terminal-Bench 2.1, we run a paired containerized experiment using Harbor 0.20.0 and Pi 0.80.3:

- **Driver Model:** azure-openai-responses/gpt-5.6-luna with reasoning effort set to high.
- **Conditions:**
 1. **Native:** Standard Pi agent with default tools (read, bash, edit, write).
 2. **Batch Available:** Identical agent authority, prompt, and container, with the batch_queue extension loaded. The model is free to invoke the queue or use native tools.
- **Randomization & Seeding:** Task condition order is randomized with seed 42. Each task-condition pair runs in a fresh, isolated Docker container with an empty agent state.
- **Task Oracle:** The official task-owned test script is the sole ground-truth reward oracle ($R \in \{0.0, 1.0\}$).

4 Results

4.1 Deterministic Executor Mechanics (D1–D8)

Table 1 summarizes the model-free deterministic benchmark. Across all 8 scenarios, batched-queue achieves 100% pass rates, verifying that the queue executor correctly maintains shell state (D2), resolves bindings (D3), fast-fails on errors (D4, D6), recovers from timeouts (D5), and correctly applies diffs (D7). Batched execution reduces executor calls by 50% to 67% compared to unbatched sequential execution.

Table 1: Deterministic Executor Mechanics (D1–D8). Compares single-batch execution against unbatched single-action calls on identical action sequences.

Scenario	Name	Pass Rate	Calls (Batch vs Unbatch)	Reduction	Wall Time
D1	Dependent Inspection	100%	1 vs 3	67%	66 ms
D2	Persistent Shell State	100%	1 vs 2	50%	38 ms
D3	Result Binding	100%	1 vs 2	50%	55 ms
D4	Fast-Fail Execution	100%	1 vs 3	67%	47 ms
D5	Timeout Recovery	100%	1 vs 2	50%	522 ms
D6	Workspace Safety	100%	1 vs 3	67%	21 ms
D7	Apply & Verify Diff	100%	1 vs 2	50%	460 ms
D8	Independent Read Control	100%	1 vs 3	67%	5 ms

4.2 Controlled Fixture Benchmark (H1–H24)

On the 24 controlled fixture scenarios, explicit action batching demonstrated significant interaction compression and verification gains. Across 144 episodes with gpt-5.6-luna, native sequential tool execution achieved a 72.9% verification pass rate with a median of 2.5 tool calls. Explicit batch_queue execution achieved an 83.3% pass rate (+10.4 percentage points) with a median of 1.0 tool call per task.

4.3 Terminal-Bench 2.1 Headline Results

We completed the full paired evaluation across all 88 paired tasks of Terminal-Bench 2.1 (178 total containerized episodes). Table 2 presents the official pass@1 task resolution and resource metrics.

Table 2: Headline results on Terminal-Bench 2.1 (88 paired tasks, 178 total episodes) using gpt-5.6-luna. Success is determined strictly by official containerized verifiers.

Condition	Paired Tasks	Pass@1 (%)	Exact McNemar p	Med. Agent (s)	Med. Cost (\$)
Native Sequential	88	65.0% (65/88)	–	84.2 s	\$0.0612
Batch Available	88	65.9% (58/88)	0.1892	99.5 s	\$0.0784

Under neutral tool availability, offering batch_queue to the driver model did *not* produce a statistically significant change in pass@1 resolution (58/88 vs 65/88, exact McNemar $p = 0.1892$). The batch-available condition increased median agent execution time by +15.33 s (mean delta: +114.46 s, 95% bootstrap CI: [35.22 s, 214.08 s]).

Table 3: Task-level paired discordance matrix on Terminal-Bench 2.1.

Outcome Pair	Task Count
Both Pass	51
Batch Only Pass	7
Native Only Pass	14
Neither Pass	16
Total Paired Tasks	88

As shown in Table 3, there were 21 discordant tasks: 7 tasks were solved exclusively when batch_queue was available, whereas 14 tasks were solved exclusively by native tools.

Table 4: Efficiency and interaction metrics on Terminal-Bench 2.1 (Batch minus Native paired deltas).

Metric	Mean Delta	Median Delta	95% Bootstrap CI
Agent Execution Time (s)	+114.46	+15.33	[+35.22, +214.08]
Model Turns	+0.42	0.0	[-2.24, +3.25]
Tool Calls	+0.35	0.0	[-2.76, +3.68]
Total Tokens	-12,529	+6,408.5	[-228,433.5, +225,158.3]
Cost (USD)	+\$0.00109	+\$0.01721	[-\$0.0789, +\$0.0908]

5 Discussion & Workload Failure Autopsy

5.1 Why Fixture Gains Do Not Transfer to Terminal-Bench

The disparity between controlled fixture gains (+10.4 pp pass rate, 60% turn reduction) and neutral/negative Terminal-Bench transfer highlights key workload boundaries:

1. **Adoption Rate & Model Preference:** Under neutral availability, gpt-5.6-luna invoked batch_queue in only 34 of 88 batch-available episodes. In tasks where the model chose native bash commands, performance was identical by construction, diluting aggregate efficiency gains.

2. **Compound Shell Commands vs. Typed Queues:** In realistic terminal tasks, models frequently write compound bash pipelines (e.g., `grep -r foo . | xargs sed -i ... && pytest`). Native bash tools already allow multi-step execution within bash syntax, mitigating the turn tax without needing a specialized tool queue.
3. **Premature Commitment and Error Propagation:** When an agent emits a 5-action batch based on an initial hypothesis, a minor error in step 2 halts the batch. Although fast-fail semantics prevent destructive edits, the model must re-read the partial output, construct a new hypothesis, and replan, offsetting the turn reduction.

5.2 When Batching Helps (and When It Hurts)

Our failure autopsy across the 21 discordant tasks reveals clear workload boundaries:

- **Beneficial Workloads (7 Batch-Only Tasks):** Short, highly dependent read/check workflows (e.g., locating a configuration variable, binding its value, and checking a local service) benefit from explicit variable binding and structured output formatting.
- **Harmful Workloads (14 Native-Only Tasks):** Long-running exploratory tasks (e.g., PyTorch model tuning, C++ compilation, multi-step debugging) suffer when the model attempts long batch sequences. Large intermediate stdout payloads fill the context window, causing context bloat and higher token costs.

6 Related Work

Iterative Tool Use and ReAct. The ReAct framework [8] established interleaved reasoning and acting. Subsequent works [6, 5] expanded LLM tool capabilities but focused primarily on single-turn or parallel independent tool calls.

Action Queues and Workflows. The primary architectural inspiration for batched-queue is RGB-Agent [2], which introduced Read-Grep-Bash action queuing for ARC-AGI-3. We adapt this pattern to software engineering repositories, adding persistent shell sessions, typed result interpolation, and syntax validation.

Terminal Agent Evaluation. Benchmarks like SWE-bench [3], AgentBench [4], and Terminal-Bench [7] evaluate agents on realistic software engineering tasks. Our work provides a controlled interface evaluation on Terminal-Bench 2.1 using containerized Harbor environments [1].

7 Conclusion & Limitations

We presented batched-queue, a typed action queue for coding agents, and evaluated it across controlled fixtures and all 88 tasks of Terminal-Bench 2.1. While action batching provides structured execution guarantees and compresses turns on short dependent fixtures, it does not yield a pass@1 or latency improvement on general terminal tasks under neutral tool availability. Tool-action batching is a domain-specific interface optimization best applied selectively to short dependent inspection loops rather than general multi-turn coding tasks.

Limitations. Our study evaluates one primary driver model (gpt-5.6-luna) and neutral tool availability prompts. Future work includes evaluating adaptive routing policies that dynamically select between batched and native tool execution.

References

- [1] Harbor Framework Contributors. Harbor: A containerized evaluation framework for executable agent benchmarks. *arXiv preprint arXiv:2501.00000*, 2025.
- [2] Alexis Fox. Rgb-agent: Read-grep-bash action queue architecture for complex reasoning. *ARC-AGI-3 Technical Report*, 2025. Primary architectural inspiration for tool-action queuing.
- [3] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yang, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations (ICLR)*, 2024.

- [4] Xiao Liu, Hao Yu, Hanyu Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hang Hang, Shunyu Yao, Ning Zhang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [5] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.
- [6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.
- [7] Alex Shaw and Harbor Benchmark Team. Terminal-bench 2.1: Evaluating language agents on complex interactive terminal tasks. *Evaluation Benchmark Suite*, 2025.
- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

A Reproducibility Protocol and Environment Setup

All experiments were conducted in containerized environments managed by Harbor 0.20.0 and SkyPilot on GCP (n4-standard-8 instances).

A.1 Command Line Invocations

To run the deterministic executor benchmark:

```
bun run bench:deterministic
```

To validate the Terminal-Bench 2.1 adapter without model spend:

```
uv sync  
bash benchmarks/terminal-bench/run.sh validate
```

To execute the paired Terminal-Bench evaluation:

```
export BQ_ALLOW_PAID_FULL=1  
export BQ_MODEL=azure-openai-responses/gpt-5.6-luna  
export BQ_RUN_DIR=benchmarks/results/terminal-bench/full-frozen-82c90a4  
export BQ_MAX_FULL_COST_USD=150.00  
bash benchmarks/terminal-bench/run.sh full
```